# Course Project: Database Design and Implementation

Group 1

Andre de Breyne

Andre Ricardo Caralli Junior

Bruno de Arantes Leite Sassi

Isaac Inalegwu Agba

Master of Data Analytics, University of Niagara Falls Canada

CPSC 500: SQL Databases

Instructor: Sundus Shanef, PhD

March 23th, 2025

## 1. Project Overview

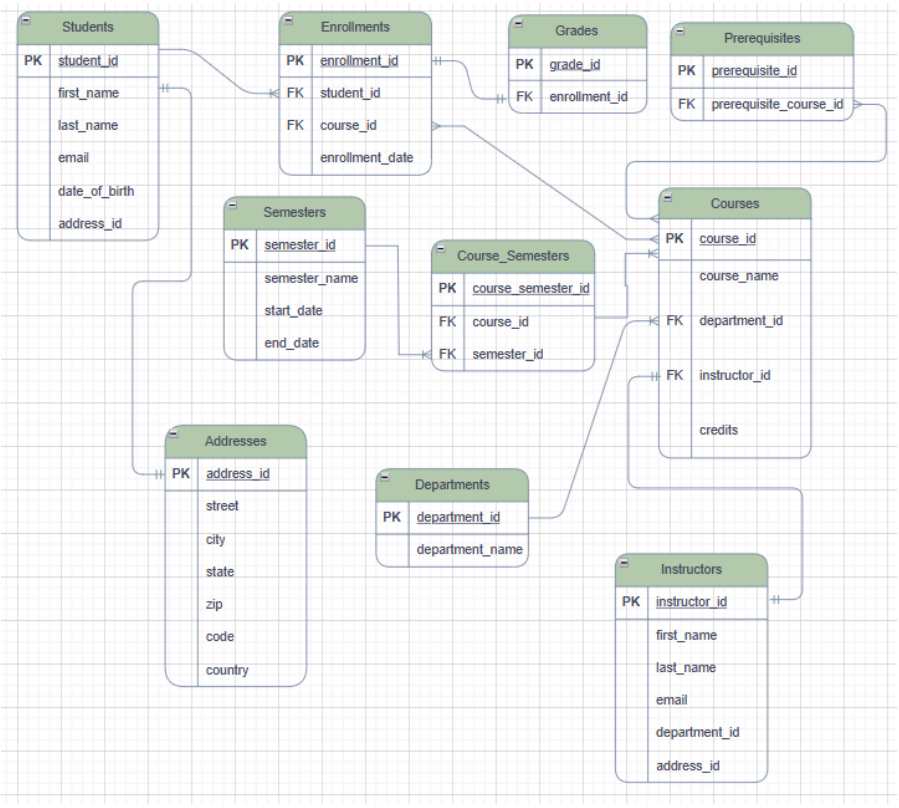
This project presents the design and implementation of a comprehensive SQL-based database system for an academic environment. The system simulates the core operations of a university, including student enrollment, course management, grading, scheduling, and instructor assignments. The database serves as both a transactional and analytical resource and is structured to support machine learning applications in educational analytics.

## 2. Entity-Relationship (ER) Diagram

An ER diagram illustrating the schema relationships is found below:

**Figure 1.**

ER diagram – University database



Source: Author’s design using Draw.io

## 3. Database Schema Design

The database is composed of interrelated tables designed to ensure normalization, eliminate redundancy, and maintain data integrity. Below is a summary of the tables:

**Table 1.**

Summary of tables in university database

| Table Name | Purpose |
|---|---|
| Addresses | Stores address information for students and instructors |
| Students | Captures student profiles and personal data |
| Instructors | Contains instructor profiles and department links |
| Departments | Academic department details |
| Courses | Course offerings and credits |
| Enrollments | Links students to courses |
| Grades | Stores letter grades per enrollment |
| Semesters | Stores semester metadata |
| Course_Semesters | Maps courses to semesters |
| Prerequisites | Defines course prerequisite relationships |

Source: Author’s compilation

## 4. Data Definition (DDL)

Each table in the database was created using CREATE TABLE statements, incorporating essential structural and integrity constraints. Primary keys were defined with the AUTO_INCREMENT attribute to ensure unique identification of records, while foreign key constraints were used to establish and enforce relationships between tables such as Students, Courses, and Departments. Additional constraints like NOT NULL and

UNIQUE were applied to maintain data quality and consistency. The design also includes composite relationships—such as the linkage between courses and semesters through a junction table—which provides the flexibility needed for managing course schedules across multiple academic terms.

Below are sample DDL statements used to create the database schema. These include CREATE TABLE definitions along with relevant constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE. These statements demonstrate how the schema enforces data integrity and establishes relationships between entities.

```
CREATE TABLE Students (
   student_id INT PRIMARY KEY AUTO_INCREMENT,
   first_name VARCHAR(50) NOT NULL,
   last_name VARCHAR(50) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   date_of_birth DATE NOT NULL,
   address_id INT,
   FOREIGN KEY (address_id) REFERENCES Addresses(address_id)
);
```

**5. Data Manipulation (DML)**

The system was populated with realistic sample data to simulate a functioning academic environment. This includes 70 student records, 20 instructor profiles, and 50 distinct courses distributed across 10 departments. A total of 100 enrollment records were created, each linked to a corresponding grade, and 10 semesters were defined with 100 course offerings mapped to specific time periods. Data Manipulation Language (DML) operations were extensively used to insert records into all tables. While UPDATE and DELETE operations were tested during development to ensure data consistency and dynamic behavior, they were not included in the final script for clarity and focus.

The following examples illustrate how data was inserted, updated, and deleted using DML statements. These operations populate the tables with realistic values and simulate the dynamic nature of a real-world academic system.

```
-- INSERT SAMPLE
INSERT INTO Departments (department_id, department_name) VALUES
(1, 'Computer Science'),
(2, 'Mathematics');

-- UPDATE SAMPLE
UPDATE Students
SET email = 'updated.email@example.com'
WHERE student_id = 1;

-- DELETE SAMPLE
DELETE FROM Enrollments
WHERE enrollment_id = 70;
```

**6. Data Retrieval (DQL)**

Custom SQL queries were developed to facilitate the analysis of academic performance, course load distribution, and curriculum structure. These queries support the generation of datasets suitable for machine learning tasks. For example, the database can be queried to retrieve comprehensive student-course-grade data that can be used in predictive models. Additionally, aggregate functions allow for the calculation of average grades by course or department, providing insight into academic trends. The system also enables the identification of students potentially at risk by deriving GPA values from recorded grades. Furthermore, enrollment patterns across semesters can be tracked to analyze course popularity and student progression over time.

To extract meaningful insights from the database, a variety of DQL (Data Query Language) statements were written. These include simple SELECTs, JOINs between

multiple tables, aggregate functions, and groupings. Below are representative examples of queries used for data analysis and machine learning dataset preparation.

```
-- Joining tables SAMPLE
SELECT s.first_name, s.last_name, c.course_name, g.grade
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
JOIN Grades g ON e.enrollment_id = g.enrollment_id;

-- Aggegation SAMPLE
SELECT c.course_name, AVG(CASE grade
   WHEN 'A' THEN 4
   WHEN 'B' THEN 3
   WHEN 'C' THEN 2
   ELSE 0 END) AS average_gpa
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
JOIN Grades g ON e.enrollment_id = g.enrollment_id
GROUP BY c.course_name;
```

**7. Machine Learning Integration**

The database structure was designed with analysis-ready data in mind. Thanks to its clear relationships and normalized tables, it enables efficient extraction of datasets for supervised learning tasks. For instance, a dataset that joins Students, Enrollments, Courses, and Grades can be leveraged to build predictive models that estimate student success in future courses. Additionally, the structured representation of course prerequisites allows for graph-based optimization of course sequencing, enabling the identification of ideal learning paths or detecting bottlenecks in the curriculum.

The following sample queries demonstrate how the database can be used to generate meaningful reports for academic analysis and decision-making. These examples highlight the system’s capability to extract performance metrics, enrollment trends, and

curriculum structure, supporting both operational needs and machine learning applications.

## I. Report: Student Performance by Course

Retrieves all students and their grades for each course they've taken.

```
SELECT
   s.student_id,
   s.first_name,
   s.last_name,
   c.course_name,
   g.grade
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
JOIN Grades g ON e.enrollment_id = g.enrollment_id
ORDER BY s.student_id, c.course_name;
```

## II. Report: Average Grade per Department

Calculates the average GPA per department, assuming a grade scale (A=4, B=3, C=2).

```
SELECT
   d.department_name,
   ROUND(AVG(
      CASE g.grade
         WHEN 'A' THEN 4
         WHEN 'B' THEN 3
         WHEN 'C' THEN 2
         ELSE 0
      END
   ), 2) AS average_gpa
FROM Departments d
JOIN Courses c ON d.department_id = c.department_id
JOIN Enrollments e ON c.course_id = e.course_id
JOIN Grades g ON e.enrollment_id = g.enrollment_id
GROUP BY d.department_name;
```

## III. Report: Enrollment Trends per Semester

Counts the number of enrollments per semester.

```
SELECT
```

```
    sem.semester_name,
    COUNT(e.enrollment_id) AS total_enrollments
FROM Enrollments e
JOIN Course_Semesters cs ON e.course_id = cs.course_id
JOIN Semesters sem ON cs.semester_id = sem.semester_id
GROUP BY sem.semester_id
ORDER BY sem.start_date;
```

### IV. Report: Courses and Their Prerequisites

Displays all courses with their respective prerequisites.

```
SELECT
    c.course_name AS course,
    p.course_name AS prerequisite
FROM Prerequisites pr
JOIN Courses c ON pr.course_id = c.course_id
JOIN Courses p ON pr.prerequisite_course_id = p.course_id
ORDER BY c.course_name;
```

## 8. Extended Features

We explored several advanced database concepts to enhance the functionality and analytical capabilities of the system. The use of prerequisite relationships allows for the construction of dependency graphs, which can be applied to optimize course sequencing and academic planning. Additionally, the semester structure was normalized through the use of a junction table (Course_Semesters), enabling flexible and scalable mapping of courses to academic terms. This design also supports clean time-series analysis by allowing the tracking of student performance and grade progression across multiple semesters.

## 9. Conclusion

The project successfully demonstrates the design and implementation of a functional academic database system using SQL. With a normalized structure, referential

integrity, realistic data, and analytical potential, this system can be a solid foundation for further development or research in educational technology.

## 10. Appendices

- SQL Code: Attached as .sql file (includes DDL, DML).

- ER Diagram: [Insert link]

- Presentation: Attached .pptx file.